EcoSim

Simulation d'un écosystème proies-prédateurs

Djibril Abdoulhamidi

March 2026

Table des matières

1 Introduction **3**

2 Modèle théorique **3**

3 Modélisation algorithmique **4**
3.1 Structure du programme 4
3.2 Réglementation de la simulation 5

4 Parcours de la simulation **5**

5 Résultat et analyse **6**

6 Conclusion **8**

7 Annexes **10**
7.1 Ensemble de paramètres de configuration 10

1 Introduction

L'écosystème naturel, aussi bien la faune que la flore, n'a jamais été autant altéré qu'à ce jour par les activités humaines. La pollution de l'air, le réchauffement des océans, l'altération des sols, la chasse et d'autres facteurs seraient grandement responsables, à ce jour, de l'extinction d'environ 45 000 espèces selon l'UICN (Union Internationale de la Conservation de la Nature). Des moyens de prévention pour freiner cette menace sont mis en place. Une mesure régulière de la population est un enjeu majeur dans ces dispositifs de prévention, afin d'aborder une approche optimale face à la situation. Toutefois, les mesures collectées directement sur le terrain sont très coûteuses et sont déployées peu souvent. Une des façons de garder un œil sur l'évolution démographique d'une population se base sur des modélisations d'écosystèmes et des simulations de scénarios.

Une modélisation consiste à créer une version simplifiée d'un système réel, dans notre cas biologique, pour l'analyser et mieux le comprendre. Cette démarche nous permet également de tester des hypothèses sans réel risque. On pourrait citer comme modélisation connue celle du modèle Lotka-Volterra.

Basé sur un encadrement académique, ce projet constitue un prolongement d'une modélisation des interactions entre proies et prédateurs, sur un terrain sans propriété complexe et dépourvu d'espèces particulières. Cette étude généraliste vise à une compréhension globale des implémentations probabilistes au sein de jeux de données. Nous réviserons premièrement les fondements théoriques de notre projet. Ensuite, nous expliciterons nos choix algorithmiques pour enfin analyser nos résultats.

2 Modèle théorique

Le principal problème de ce projet réside dans le rapprochement de l'évolution de notre modèle aux oscillations observées au *Lotka-Volterra*.

Le modèle Lotka-Volterra est un système d'équations différentielles qui est utilisé en écologie pour modéliser une évolution conjointe de deux populations, dans une perspective d'évolution dynamique :

Une baisse démographique des prédateurs laisse les proies se reproduire de façon exponentielle. La surpopulation engendrée laisse les prédateurs restants s'approvisionner plus facilement, la croissance de leur population devient ainsi plus aisée.

Inversement, plusieurs prédateurs menacent une extinction des proies a priori. Toutefois, après un seuil franchi, la difficulté de trouver des proies augmente le taux de mortalité des prédateurs, et laisse les proies se multiplier plus aisément.

En somme : la croissance des proies précède toujours celle des prédateurs, et inversement. Réciproquement, la diminution des proies précède celle des prédateurs, et inversement.

Soit un environnement $\boldsymbol{E}$, un terrain de l'écosystème $\boldsymbol{ECO}$ de notre simulation. Pour des objectifs académiques, aucun relief, obstacle ou autre caractéristique d'un environnement classique, hormis les plantes, ne sera pris en compte dans notre écosystème. Toutes les plantes sont identiques et comestibles.

Soit $\boldsymbol{X}$ l'ensemble des proies et $\boldsymbol{Y}$ l'ensemble des prédateurs présents dans $\boldsymbol{E}$. Nous ne faisons pas de distinction entre les différentes proies et différents prédateurs. L'unique interaction possible dans $\boldsymbol{E}$ est la chasse et la reproduction. Toute proie peut être la cible de tout prédateur, et aucune confrontation intra-espèce ne peut être enclenchée.

Soit $\boldsymbol{T}$ notre unité de temps, variable selon les initialisations de $\boldsymbol{E}$ et selon des préférences d'interprétation. Mathématiquement, l'évolution de notre population pour tout temps $\boldsymbol{T}$ de notre modèle se traduit de la façon suivante :

$$\begin{cases} \dfrac{dx}{dt} = \alpha x - \beta xy \\ \dfrac{dy}{dt} = \delta xy - \gamma y \\ X = X + \dfrac{dx}{dt} \\ Y = Y + \dfrac{dy}{dt} \end{cases}$$

Avec :

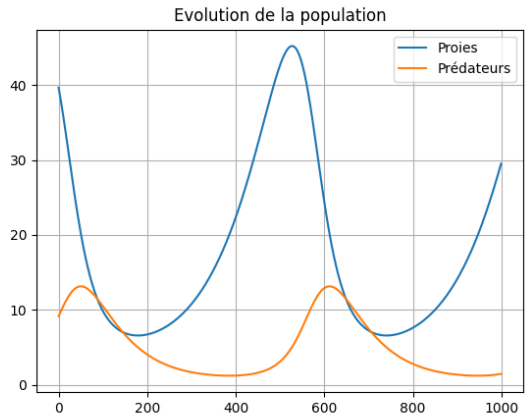
— α le taux de croissance naturel des proies

— β le taux de prédation

— γ le taux de reproduction par proie consommée

— δ le taux de mortalité naturel des prédateurs

Pour notre écosystème $\boldsymbol{E}$, prenons $\alpha = 1{,}0$, $\beta = 0{,}2$, $\gamma = 1{,}5$ et $\delta = 0{,}075$. Voici une visualisation des oscillations cibles de notre modèle :



On observe après cette figure que les populations évoluent de manière périodique. Les propriétés d'expansion-retraction mutuelle se vérifient simplement.

On remarque également que ces oscillations sont régulières et ne convergent pas vers un scénario provoquant la fin d'une ou des espèces. Cela s'explique par nos hypothèses expliquées plus haut, avec l'absence de contraintes environnementales ou de diversité au sein des espèces.

Il nous faut désormais retranscrire, sur un modèle plus réaliste, a priori dépourvu de toute intervention humaine, réalisant une évolution de la faune similaire.

3 Modélisation algorithmique

3.1 Structure du programme

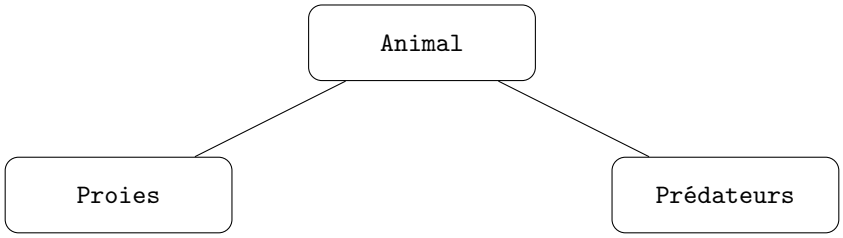
Les facteurs majeurs de notre domaine de recherche constituent notre écosystème $\boldsymbol{ECO}$, incluant la faune (les proies $\boldsymbol{X}$ et prédateurs $\boldsymbol{Y}$) et la flore (l'environnement $\boldsymbol{E}$). L'ensemble de ceci regroupé dans un seul et même écosystème. Dans ce projet, j'ai jugé bon d'opter pour une programmation orientée objet (POO) sur ***Python***, pour des raisons de simplification de codage.

Nous remarquerons dans notre définition d'$\boldsymbol{ECO}$ qu'aucune espèce n'est prédéfinie, nous ne comptons que des animaux ayant soit le titre de proies, soit celui de prédateur. Cette approche volontairement simplifiée nous permet de poser une base commune à tous les individus du modèle, regroupés sous une même abstraction : la classe `Animal`. Pour les interactions qui nous intéressent, la classe animal possède les attributs suivants : des coordonnées sur le plan (`posx`, `posy`), l'énergie `energie` dont il dispose, un indicateur binaire `est_vivant` de son état vital, initialisé à `True`, et une probabilité `p_rep` de reproduction initialisée selon l'espèce. La probabilité de reproduction est initialisée à 0,4 pour les proies et 0,2 pour les prédateurs. L'ensemble de ces éléments est initialisé avec la signature :

Signature : `__init__(self, x : int, y : int, energie : float = ENERGIE_BASE)`

Cette stratégie nous évite une duplication de code pour chaque espèce, et garantit une cohérence des fonctions et attributs pour chaque classe.

Nous concevons par la suite des héritières de la classe Animal, à savoir `Proies` et `Prédateurs`, avec seulement le système de fonctionnement de la nutrition qui diffère.



Après avoir défini les entités vivantes de notre écosystème, il nous faut modéliser les structures qui encadreront leurs interactions lors de nos simulations.

Tout d'abord, la flore $\boldsymbol{E}$ sera définie dans la classe `Environnement`. Nous cherchons ici à modéliser les ressources d'énergie pour les proies. Les plantes étant toutes les mêmes, et ayant un apport d'énergie identique pour manger, E représentera une grille (une matrice) à valeur booléenne : `True` s'il y a des plantes à l'emplacement (x, y), `False` sinon.

Signature : `__init__(self,long,larg)`

Une fois la flore définie, le monde regroupant ces éléments est défini à travers la classe `Monde`. Cette dernière contiendra les dimensions de notre écosystème (la longueur `long` et la largeur `larg`), une liste `liste_animaux` contenant l'ensemble des animaux de notre écosystème ($\mathbf{X} \cup \mathbf{Y}$), l'environnement `env` de notre écosystème $\boldsymbol{E}$ et un historique des quantités de proies et prédateurs `hist_pro_pre` de notre simulation.

Signature : `__init__(self,long,larg)`

3.2 Réglementation de la simulation

Notre écosystème simplifié fait rentrer en contact deux uniques espèces avec des capacités d'interactions contraintes : nous ne retrouverons que la chasse pour les prédateurs et la reproduction intra-espèces. Pour en revenir à notre classe `Animal`, elle contient en son sein l'ensemble des actions dont les proies et les prédateurs ont besoin, sauf l'action `manger`, qui est définie pour chaque espèce de façon différente. Selon les interactions ciblées, nous donnerons à notre animal le déplacement vers une direction générée aléatoirement, la détection de voisins de son entourage dans une surface d'un carré 3 x 3, la tentative d'approvisionnement.

Un fichier config.py portera l'ensemble des valeurs et probabilités que nous voulons attribuer à notre écosystème, tel que le gain d'énergie pour une plante consommée, la probabilité de réussir sa chasse et autre.

4 Parcours de la simulation

La classe `Simulation` représente le moteur de ce projet, faisant tourner notre écosystème test. Notre simulation prend notre monde pré-modélisé pour parcourir chaque animal y étant présent et faire suivre à ce dernier la chaîne suivante :

Se déplacer – ? –> Tenter de manger –> Tenter de se reproduire

En premier lieu, vient le déplacement. Qu'il soit mort ou non, cette action n'implique aucune incohérence majeure à notre simulation. Cette action est donc appliquée obligatoirement à chaque animal avant d'itérer la suite de notre parcours (nous n'aurons qu'à dire que divers facteurs non perceptibles comme le vent, la pluie, des animaux les déplaçant pour vérifier s'ils sont morts, sont à l'origine du déplacement de la carcasse).

Après cela, nous vérifions l'état vital de notre animal avec la fonction `est_vivant(self)` pour savoir s'il est éligible à démarrer la suite du parcours, d'où le point d'interrogation dans le schéma. La tentative d'approvisionnement ne revient uniquement à faire appel à la fonction manger de l'animal et de savoir s'il y a eu consommation ou non. Une cumulation maximale d'énergie sera fixée à `MAX_ENERGIE`, que l'on initialisera selon les scénarios voulus (basse entraîne une extinction rapide des prédateurs et une hausse exponentielle des proies, haute laisse plus de temps aux prédateurs de se multiplier, et des oscillations plus concurrentes). Pour ce qui est de la faculté de manger pour chaque espèce :

— **Le prédateur** :
  - Il recherche parmi ses voisins des proies potentielles,
  - Il tente de chasser chaque proie voisine et ne s'arrête pas tant qu'il n'a pas exploré toutes ces proies, et qu'il n'a pas pu en chasser une. Aucune perte d'énergie par tentative.

— **La proie** :
  - Il recherche si des plantes ne sont pas disponibles à sa position,
  - Si oui, il les mange obligatoirement, en dépit de l'énergie dont il dispose déjà.

Enfin, après cette action, l'animal tentera de se reproduire avec un autre individu de la même race. Aucun genre n'a été défini dans cet écosystème, nous simulons un système de reproduction sexuée avec hermaphrodisme, tels les escargots ou les vers de terre. Nous vérifions en premier lieu si l'animal est éligible à la reproduction avec un seuil d'énergie `MIN_REPRODUCTION`. Après confirmation de son éligibilité, l'animal recherchera parmi ses voisins des individus de la même espèce que lui, et une fois le repérage fini, les tentatives de reproduction suivront une loi de probabilité géométrique, avec une probabilité p = `p_rep`, et pour nombre d'essais maximum le nombre total de congénères voisins dont il dispose ; une tentative de reproduction par voisin. Notons qu'aucune perte d'énergie n'est à signaler par tentative de reproduction. S'il y a eu un nouveau-né, nous l'ajoutons à notre liste de nouveau-nés `animal_nouv`, préalablement initialisée vide avant le démarrage de la simulation. Nous ne l'ajoutons pas directement à la liste pour éviter des erreurs sur des changements de variables en pleine boucle.

Une fois les trois étapes passées, nous rafraîchissons notre écosystème pour l'itération qui suit. Premièrement, nous rajoutons l'ensemble des nouveaux-nés `animal_nouv` obtenus de notre simulation dans notre liste d'animaux, avec la supposition qu'ils auront déjà atteint l'âge adulte à la prochaine itération. Nous nous débarrassons de ceux n'ayant pas survécu à la simulation, soit ceux avec un indicatif `est_vivant` à `False`. Ensuite, nous parcourons chaque case de la grille de notre environnement et nous vérifions s'il y a une pousse ou non. Le cas échéant, nous soumettons la repousse de la plante à une probabilité `P_POUSSE_PLANTES`, non impactée dans le temps. Enfin, nous actualisons notre historique `hist_pro_pre`.

5 Résultat et analyse

Pour rappel, notre objectif est de nous approcher au possible d'une oscillation style Lotka-Volterra, où les populations varient en fonction de l'autre de façon périodique et dynamique. De la formule du système d'équations, nous disposons de quatre paramètres α, β, γ, et δ responsables de la périodicité de la courbe.

Notre approche ne donne pas directement des valeurs à ces paramètres mais donne plutôt des paramètres de configuration responsables des potentielles valeurs de ces dernières. Nous noterons notamment :

— La croissance des proies α par itération dépend de leur éligibilité à la reproduction, soit l'énergie dont elles disposent. Plus il y a de plantes à disposition, plus les proies mangent, plus elles ont de la chance d'être au-dessus du seuil de reproduction. Alpha est donc dépendante de plusieurs variables, dont les plus importantes sont `NB_PROIES`, `P_REPRODUCTION_PROIES`, `MIN_REPRODUCTION` et `P_POUSSE_PLANTES`.

$$\alpha \sim f(\text{ressources} + \text{facilité de reproduction})$$

— Le taux de prédation inclut la probabilité qu'un prédateur rencontre une proie dans son entourage. La densité des proies dans l'écosystème joue grandement dans ce dernier. Nous noterons que cette densité dépend du nombre de proies présentes ainsi que des dimensions de notre écosystème. Les variables principales responsables de β sont `MONDE_LONGUEUR`, `MONDE_LARGEUR`, `NB_PROIES`, `NB_PREDATEUR` et `P_PREDATION`.

$$\beta \sim f(\text{P_PREDATION} + \text{densité})$$

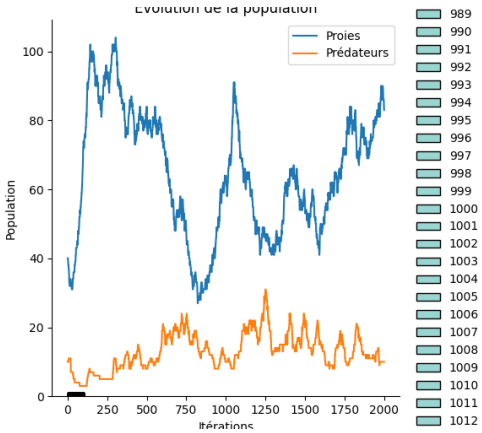
— Le taux de reproduction des prédateurs par proie consommée, dans notre écosystème, n'est pas un terme approprié. La reproduction des prédateurs ne dépend uniquement de l'énergie cumulée pour passer l'éligibilité. Manger apporte certes de l'énergie en plus, mais un prédateur n'ayant pas mangé peut rester en droit de se reproduire dans notre modélisation. Un terme approprié serait le taux de reproduction des prédateurs. Ce dernier dépend grandement, une nouvelle fois, de la densité de ces derniers, ainsi que de l'énergie dont ils disposent. Les variables responsables sont `NB_PREDATEUR`, `GAIN_NOURRITURE`, `MIN_REPRODUCTION` et `P_REPRODUCTION_PREDATEURS`.

$$\delta \sim f(\text{ ressources} + \text{facilité reproduction})$$

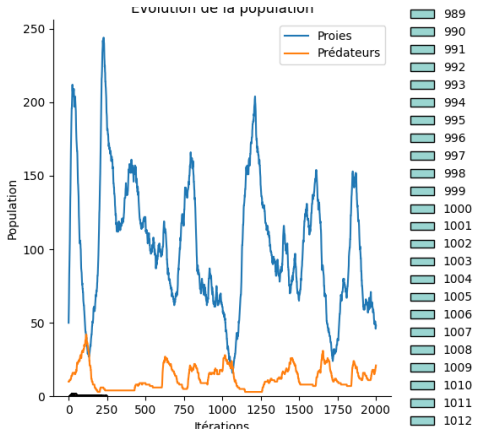
— La mortalité des prédateurs inclut tout ce qui contribue à leur perte d'énergie, seul moyen pour eux d'atteindre la mort. Les déplacements obligatoires dans chaque itération ainsi qu'une densité faible des proies sont responsables de la baisse démographique des prédateurs. Les configurations responsables de ces paramètres sont `COUT_DEPLACEMENT`, `COUT_REPRODUCTION` ainsi que les autres variables jouant sur la densité des proies.

$$\gamma \sim f(\text{coûts énergétiques, accès nourriture})$$

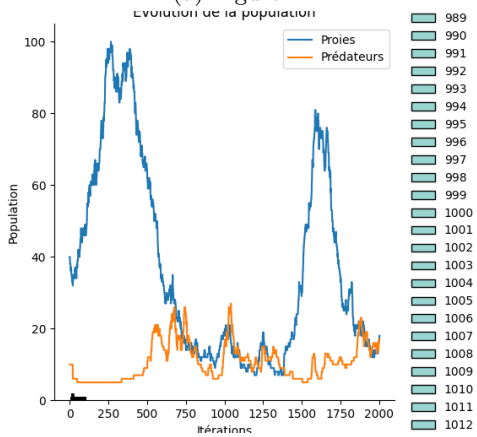
Voici différents résultats d'oscillations obtenus sur différents mondes test générés aléatoirement, sur 2000 itérations.



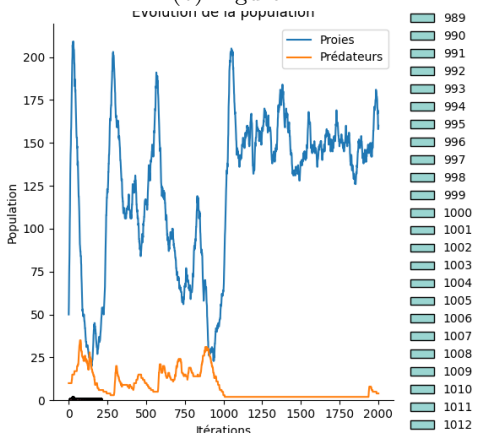
(a) Figure 1



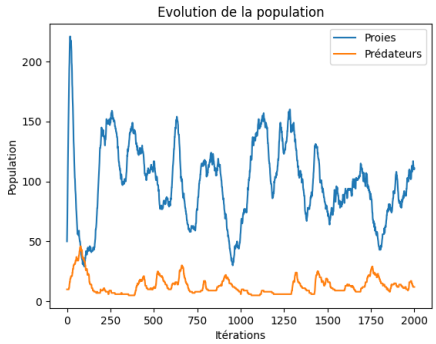
(b) Figure 2



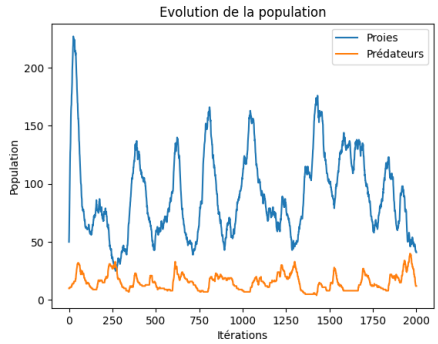
(c) Figure 3



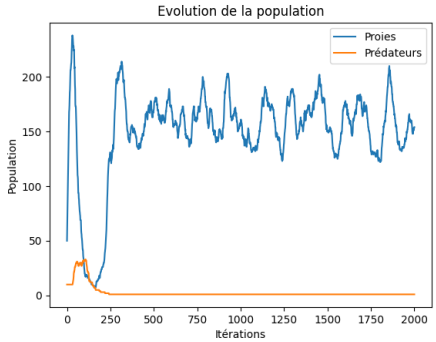
(d) Figure 4



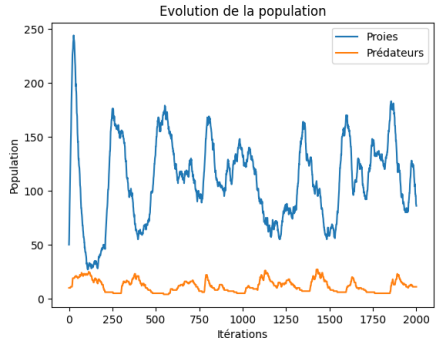
(a) Figure 5



(b) Figure 6



(c) Figure 7



(d) Figure 8

Nous constatons dans les dernières images un rapprochement plus conséquent de notre modèle théorique. Plusieurs constats sont à établir pour justifier nos oscillations. D'emblée, nos pseudo-variables du modèle théorique ne sont pas fixes et changent selon les circonstances de la simulation. Une implémentation identique serait quasi-irréaliste, malgré notre écosystème simplifié au possible. Autre remarque, les mondes ci-simulés n'ont pas été générés par un humain, mais de façon complètement aléatoire par notre machine. Des graphes aux données aberrantes tels que les figures 4 et 7, indiquant une extinction des prédateurs, sont tout à fait des résultats auxquels nous pouvons nous attendre.

Outre ces détails, les variations dépendantes et dynamiques sont perceptibles et à peu près conformes à notre modèle théorique. Les résultats obtenus montrent une évolution oscillatoire des populations de proies et de prédateurs. Dans un premier temps, la population de proies augmente en raison de leur reproduction, ce qui permet ensuite une augmentation du nombre de prédateurs grâce à une nourriture plus abondante. Cette augmentation des prédateurs entraîne alors une diminution des proies, suivie d'une baisse de la population de prédateurs.

De plus, dans certaines simulations, l'extinction d'une des populations peut survenir, ce qui ne correspond pas au modèle théorique. Ces différences s'expliquent par la prise en compte de l'espace, des interactions locales et du hasard dans notre implémentation.

6 Conclusion

Ce projet avait pour objectif de modéliser et simuler un écosystème simplifié mettant en interaction deux populations : des proies et des prédateurs, dans le but de retrouver des dynamiques proches de celles décrites par le modèle de Lotka–Volterra.

Dans un premier temps, nous avons étudié le cadre théorique de ce modèle, basé sur un système d'équations différentielles décrivant l'évolution couplée des deux populations. Ce modèle met en évidence des oscillations périodiques idéales, reposant sur des hypothèses simplificatrices telles que l'absence de contraintes spatiales et de hasard.

Nous avons ensuite proposé une modélisation algorithmique basée sur une approche orientée objet. Chaque individu est représenté explicitement par une entité autonome évoluant dans un environnement discret. Les interactions reposent sur des règles locales simples : déplacement aléatoire, consommation de ressources, reproduction conditionnée par l'énergie et prédation. Cette approche permet d'introduire des éléments absents du modèle théorique, tels que la spatialisation, les interactions locales et une composante probabiliste.

Les résultats obtenus montrent globalement l'apparition d'oscillations des populations, ce qui confirme la capacité du modèle à reproduire qualitativement les comportements attendus. On observe notamment que la croissance des proies précède celle des prédateurs, et que leur diminution entraîne ensuite une baisse de ces derniers, conformément au modèle de Lotka-Volterra.

Cependant, des écarts notables apparaissent. Les oscillations ne sont pas parfaitement régulières et présentent des fluctuations dues au caractère discret et aléatoire de la simulation. De plus, certaines configurations conduisent à l'extinction d'une ou des deux populations, phénomène absent du modèle théorique. Ces différences s'expliquent par la variabilité des paramètres au cours du temps, la dépendance aux conditions initiales ainsi que par la prise en compte de l'espace et des interactions locales.

Ainsi, cette étude met en évidence l'intérêt des simulations à base d'agents pour enrichir les modèles théoriques, en apportant un réalisme supplémentaire au prix d'une perte de régularité et de prédictibilité. Elle souligne également les limites des modèles continus classiques face à des systèmes plus complexes et plus proches des conditions réelles.

7 Annexes

7.1 Ensemble de paramètres de configuration

`MONDE_LARGEUR : int = 30`	Largeur du monde
`MONDE_LONGUEUR : int = 30`	Longueur du monde
`NB_PROIES : int = 50`	Nombre d'herbivores au départ
`NB_PREDATEUR : int = 10`	Nombre de carnivores au départ
`MAX_STEPS = 2000`	Nombre maximal d'itérations de notre simulation
`ENERGIE_BASE : float = 50.0`	Énergie initiale donnée à chaque animal
`MAX_ENERGIE : float = 100.0`	Énergie maximale possible à cumuler
`COUT_DEPLACEMENT : float = 1.0`	Coût énergétique d'un déplacement
`GAIN_NOURRITURE : float = 3.0`	Gain d'énergie après avoir mangé
`MIN_REPRODUCTION : float = 28.0`	Énergie minimale pour se reproduire
`COUT_REPRODUCTION : float = 18.0`	Coût d'énergie à la reproduction
`P_PREDATION : float = 0.25`	Probabilité qu'un prédateur chasse sa proie
`P_REPRODUCTION_PROIES : float = 0.4`	Probabilité de reproduction pour deux proies
`P_REPRODUCTION_PREDATEURS : float = 0.2`	Probabilité de reproduction pour deux prédateurs
`P_POUSSE_PLANTES : float = 0.08`	Probabilité qu'une plante pousse